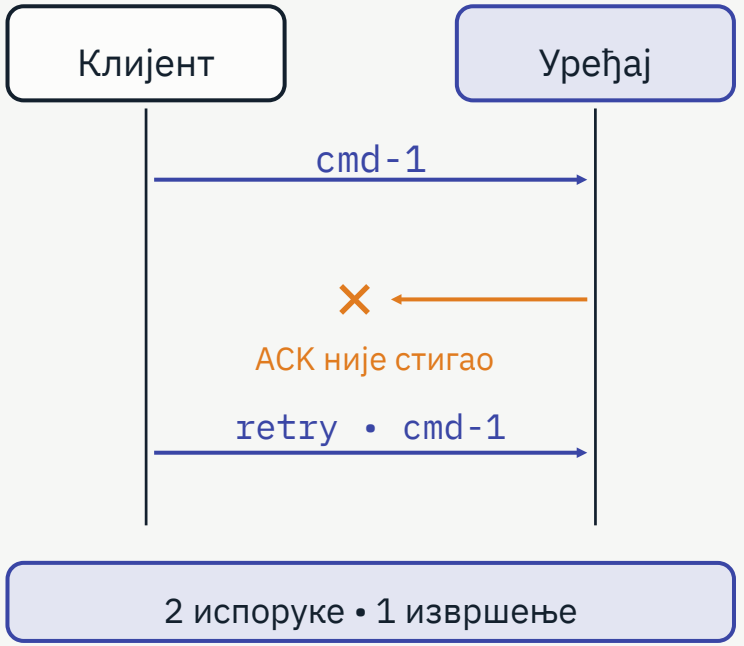
ПРИМЕЊЕНО СОФТВЕРСКО ИНЖЕЊЕРСТВО

ОСНОВЕ ДИСТРИБУИРАНОГ ПРОГРАМИРАЊА

Команде, послови и идемпотентност

Поуздан напредак без сигурне потврде



Факултет техничких наука · Универзитет у Новом Саду

Нови Сад

ПРОВЕРА

Садржај презентације

01

Појмови и границе

Кључни појмови, одговорности компоненти и конкретан failure сценарио.

03–10

02

Дистрибуирани ток

Шема тока и гаранције које повезују компоненте система.

11–13

03

Код и failure тестови

Покретање примера, доменско правило, тест и видљиви исходи.

14–19

04

Примена и провера

Принципи, замке, задатак уз пример и примена на пројектну целину.

20–27

ПРВИ ДЕО

01

Појмови и границе

Кључни појмови, одговорности компоненти и конкретан failure сценарио.

01 · ПОЈМОВИ

Тема и проблем

Разумети како се команда и дуготрајан посао воде кроз retry, преузимање и касне одговоре.

Слање захтева не значи да је удаљена радња завршена. Поновни покушај може помоћи, али може и да понови пословни ефекат.

НАПОМЕНА

Идемпотентност чува значење поновљеног захтева, а ограничено власништво над послом спречава сукоб workera.

01 · ПОЈМОВИ

Четири појма теме

Команда је захтев за промену света, а не обичан позив методе.

01 **Захтев**

валидирана намера

02 **Извршење**

рад у позадини

03 **Потврда**

може каснити

04 **Исход**

једном забележен резултат

01 · ПОЈМОВИ

Захтев

Захтев изражава пословну намеру и добија стабилан идентитет операције. Тај идентитет повезује поновљени покушај са истом намером, а не са новом радњом.

01 Носи стабилан идентитет операције, не случајан број.

02 Поновљени покушај се препознаје као иста намера.

03 Основа за идемпотентно руковање.

01 · ПОЈМОВИ

Извршење посла

Дуготрајан посао може чекати ресурс, променити власника или бити преузет након пада workera. Његов статус мора остати видљив независно од једног процеса.

01 Статус видљив независно од једног процеса.

02 Може бити преузет након пада workera.

03 Два workera не смеју веровати да су оба власници.

Касна потврда

Потврда може стићи након истека рока или након новог покушаја. Систем мора проверити да ли се односи на актуелно стање пре него што га измени.

01 Може стићи после истека рока или новог покушаја.

02 Проверава се према актуелном стању пре примене.

03 Не сме тихо преписати новије стање.

Јединствен исход

Идемпотентност чува један пословни исход за исту намеру, чак и када порука стигне више пута. Она не значи да се свака радња може безбедно понављати.

01 Један пословни исход без обзира на број покушаја.

02 Не значи да је свака радња безбедна за retry.

03 Retry има границу, не изводи се бесконачно.

Пример: поновљен захтев за исту радњу

Клијент не добија одговор на време и шаље исти захтев поново.

01 Поновни покушај користи исти идентитет операције када намера корисника није промењена.

02 Worker који преузима посао има ограничено време власништва и не може завршити рад након истека.

03 Касна потврда не сме преписати новије, завршено стање.

ДРУГИ ДЕО

02

Дистрибуирани ток

Шема тока и гаранције које повезују компоненте система.

02 · ТОК

Шема поуздане команде

Retry повећава шансу да се радња заврши, док идемпотентност чува њено значење.

Елемент	Улога у току
Захтев	јединствена намера
Ред или посао	обрада у позадини
Lease	привремено власништво workera
Исход	једном забележен резултат

02 · ДИЈАГРАМ

Изгубљена потврда не ствара нову намеру

Поновни покушај користи исти CommandId, а прималац препознаје већ извршену команду.



ТРЕЋИ ДЕО

03

Код и failure тестови

Покретање примера, доменско правило, тест и видљиви исходи.

Покретање примера

Пример води команду ка удаљеном уређају кроз неизвесност.

terminal · Корен Git репозиторијума nastava.

```
cd "03 Osnove Distribuiranog Programiranja/04 Primeri/06 Komande idempotentnost"
dotnet run --project src/Odp.Vezba06.ConsoleUi
dotnet test Odp.Vezba06.sln
```

Контекст сценарија

Правило у коду

Понављени захтев са истим CommandId не шаље команду други пут.

ПРИКАЗАНИ ПРИМЕР

Потврда помера Sent или TimedOut у Acknowledged, док поновљена потврда добија AckDuplicate.

Правило у коду

Потврда помера Sent или TimedOut у Acknowledged, док поновљена потврда добија AckDuplicate.

C# · доменско правило

```
public (DeviceCommand Command, string Code) Acknowledge(DeviceCommand command) =>
    command.State switch
    {
        CommandState.Sent =>
            (command with { State = CommandState.Acknowledged }, CommandCodes.Acknowledged),
        CommandState.TimedOut =>
            (command with { State = CommandState.Acknowledged }, CommandCodes.LateAckReconciled),
        _ => (command, CommandCodes.AckDuplicate)
    };
```

Тест који задаје квар

После истека се шаље иста команда, уређај је прима два пута, а извршава једном.

C# · NUnit

```
1   [Test]
2   public void Run_WhenAckIsMissing_ResendsTheSameCommandAndTheDeviceExecutesItOnce()
3   {
4       _dispatch.Dispatch("cmd-1", "antena-1", "rotate:120");
5       _clock.Advance(TimeSpan.FromSeconds(12));
6       var outcomes = _retry.Run();
7       using (Assert.EnterMultipleScope())
8       {
9           Assert.That(outcomes, Is.EqualTo(new[] { new RetryOutcome("cmd-1", CommandCodes.Retried) }));
10          Assert.That(_device.Deliveries, Is.EqualTo(2));
11          Assert.That(_device.ExecutedCommandIds, Is.EqualTo(new[] { "cmd-1" }));
12      }
13  }
```

Испис демонстрације

Потврда која стигне после одустајања исправља стање (late_ack_reconciled).

console · исходи демонстрације

```
operator šalje komandu -> command_sent (pokušaj 1)
operator ponavlja isti zahtev -> command_already_dispatched (pokušaj 1)
potvrda nije stigla 12 s -> cmd-7: command_retried
potvrda nije stigla još 12 s -> cmd-7: attempts_exhausted
zakasnela potvrda -> late_ack_reconciled
ista potvrda ponovo -> ack_duplicate
Uređaj je primio 2 isporuke, a izvršio 1 komandu.
```

ЧЕТВРТИ ДЕО

04

Примена и провера

Принципи, замке, задатак уз пример и примена на пројектну целину.

04 · ПРИМЕНА

Принципи и замке

Принцип важи тек када је препозната и замка која га тихо поништава.

ПРИНЦИПИ

- Статус команде описује знање система, не нагађање о удаљеном свету.
- Idempotency кључ везује поновљене покушаје за исти пословни резултат.
- Lease има рок и штити систем од закашњелог рада претходног власника.

ЗАМКЕ

- Retry без границе и без разлога за престанак.
- Две обраде које обе верују да су власници посла.
- Касна потврда која преписује завршено стање.

04 · ПРИМЕНА

Типичне грешке

Ове грешке се најчешће понављају када се принцип површно примени.

01 Retry без границе.

02 Два workera верују да су власници.

03 Касна потврда преписује ново стање.

04 · ПРИМЕНА

Како се принцип примењује

Кораци показују како се принцип препознаје и доследно спроводи у развоју.

01 Раздвојити пријем, обраду и приказ исхода.

02 Везати понављање за идентитет операције.

03 Тестирати пад workera и касну потврду.

04 · ПРИМЕНА

Питање за дискусију

Одговор се образлаже failure сценаријем и тестом, а не описом срећног пута.

Која радња би имала неприхватљиву последицу ако се изврши два пута и шта је њен стабилни идентитет?

ЗА ВЕЖБУ

Која би последица дуплог извршења била неприхватљива и који идентитет операције спречава такав исход?

04 · ПРИМЕНА

Задатак уз пример

Проширење примера уз вежбу, правило иде у домен, а доказ у тест.

01 Додати одлагање између покушаја (RetryDelay) које расте са бројем покушаја.

02 Правило додати у CommandLifecycle и тестом показати да се поновно слање не дешава пре истека одлагања.

04 · ПРИМЕНА

Контролна тачка P3 (07.04.)

Трећа контролна тачка (Вежба 5–6) заокружује основни ниво система и поставља базну линију тестова пред следећу фазу рада.

01 Кључни use-case-ови имају тестове за успешне, негативне и failure сценарије.

02 Извештај о покривености је прегледан.

03 Најмање један bug је најпре репродукован тестом уз FailureSimulator.

04 Стабилна верзија језгра је означена Git tag-ом manual-core-baseline.

04 · ПРИМЕНА

Резиме

Тим раздваја пријем захтева, обраду у позадини и приказ исхода.

Retry повећава шансу да се радња заврши, док идемпотентност чува њено значење.

НАПОМЕНА

Наредна вежба: Рад без везе и поуздана испорука.